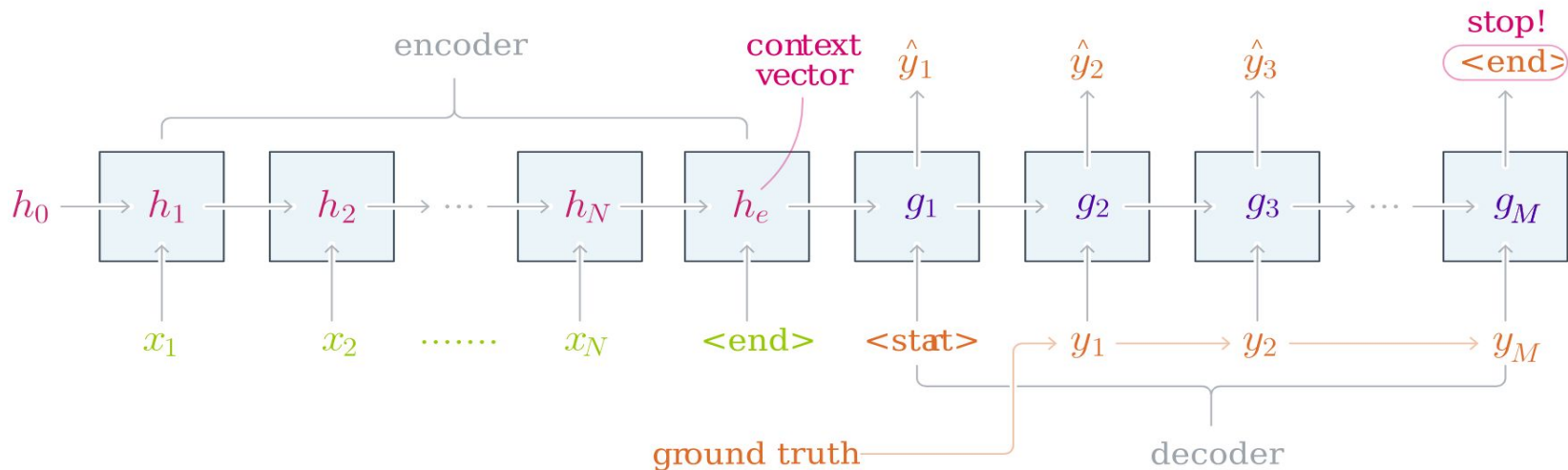


Attention и трансформеры

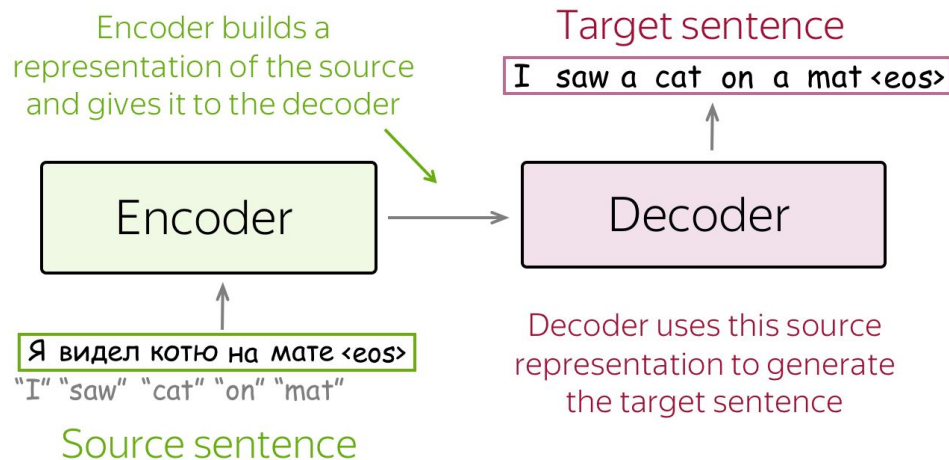
Повторение: sequence-to-sequence

Seq2seq-нейросети состоят из двух частей: энкодера (кодировщика) и декодера (декодировщика). Для обоих этих компонентов обычно берут рекуррентные нейросети.

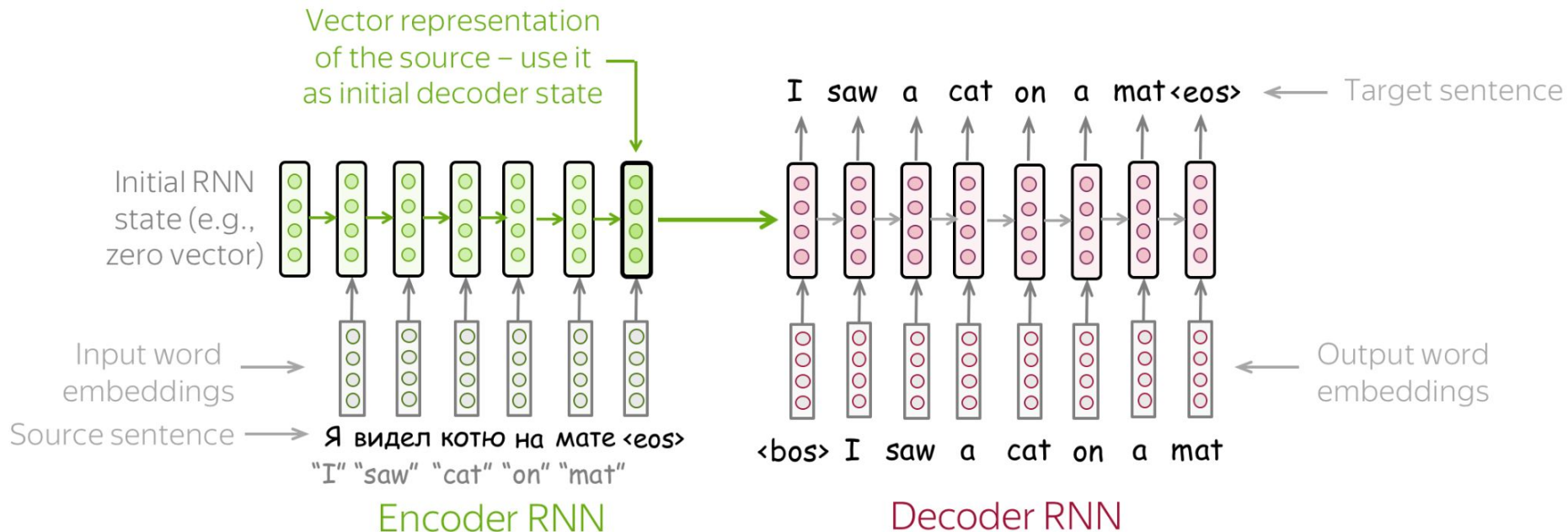


Повторение: sequence-to-sequence

- Энкодер принимает исходное предложение и агрегирует информацию из него;
- Декодер последовательно генерирует выходное предложение на основе информации из энкодера.



Seq2Seq: обучение



Teacher forcing

При генерации следующего токена декодер должен смотреть на предыдущий сгенерированный токен. Однако в начале обучения нейросети предыдущий сгенерированный токен, скорее всего, будет бессмысленным. Поэтому при тренировке seq2seq-нейросетей применяется **teacher forcing**: декодеру на вход подаются токены из заранее известной target-последовательности, которая должна быть сгенерирована “в идеале”.

Как правило, по мере обучения нейросети teacher forcing начинает применяться реже.

Subword tokenization

Для токенизации текстов в современных моделях чаще всего применяется Byte Pair Encoding (BPE): метод сегментации слов, который делит редкие слова на символные нграммы, а частотные слова оставляет целыми.

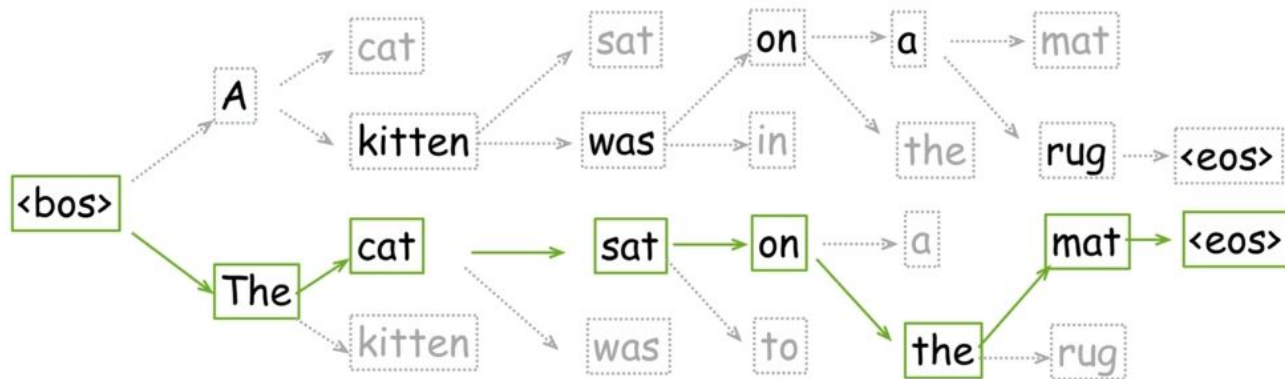
BPE также является обучаемым методом.

Categories	Examples
English text	Generative Pre-trained Transformer 4 (GPT-4) is a multimodal large language model created by OpenAI, and the fourth in its series of GPT foundation models.
English subwords	Gener ative Pre -trained Transformer 4 (G PT - 4) is a multim odal large language model created by Open AI , and the fourth in its series of G PT foundation models .
Chinese text	您可以使用下面的工具了解语言模型如何对一段文本进行标记化，以及这段文本中的标记总数。
Chinese subwords	您 可 以 使 用 下 面 的 工 具 了 解 语 言 模 型 如 何 对 一 段 文 本 进 行 标 记 化 ， 以 及 这 段 文 本 中 的 标 记 总 数 。

Beam search

Проблема: на каждом этапе генерации мы пытаемся выбрать из всех возможных токенов тот, который имеет наибольшую вероятность быть следующим. Однако, выбор самого вероятного следующего токена для конкретного предыдущего токена не гарантирует составления лучшей выходной последовательности.

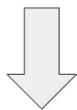
Решение: beam search - каждый раз мы будем сохранять несколько гипотез возможной выходной последовательности.



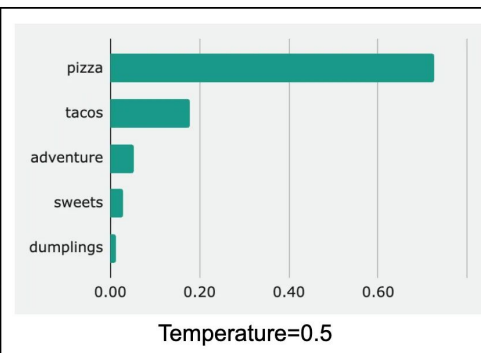
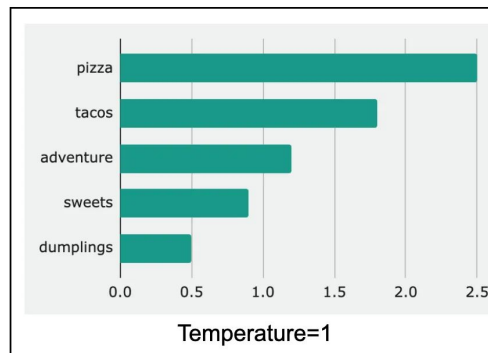
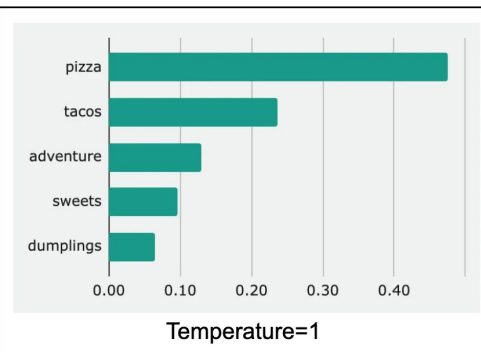
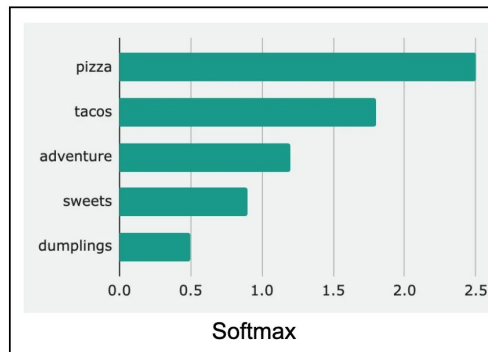
Температура

- Температура - это параметр, добавляемый к функции softmax, позволяющий управлять “креативностью” генерации.

$$p_i = \text{Softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^n \exp(z_j)}$$

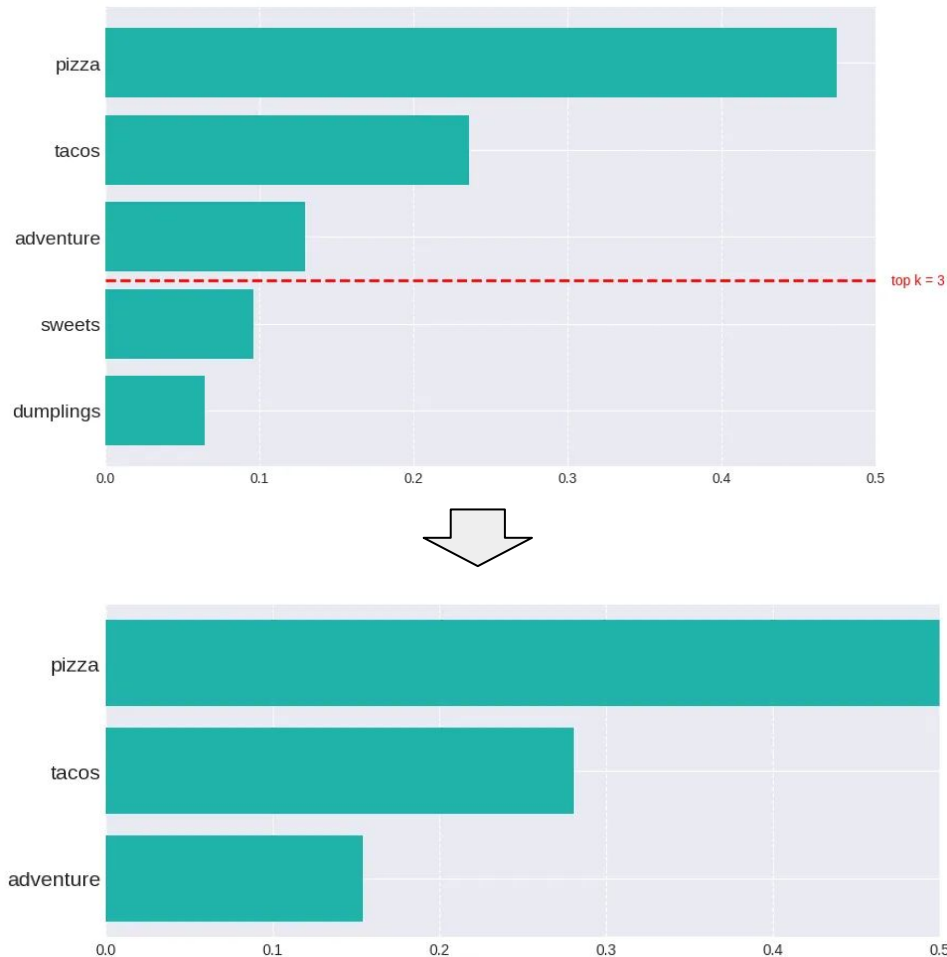


$$p_i(T) = \text{Softmax}(z_i/T) = \frac{\exp(z_i/T)}{\sum_{j=1}^n \exp(z_j/T)}$$



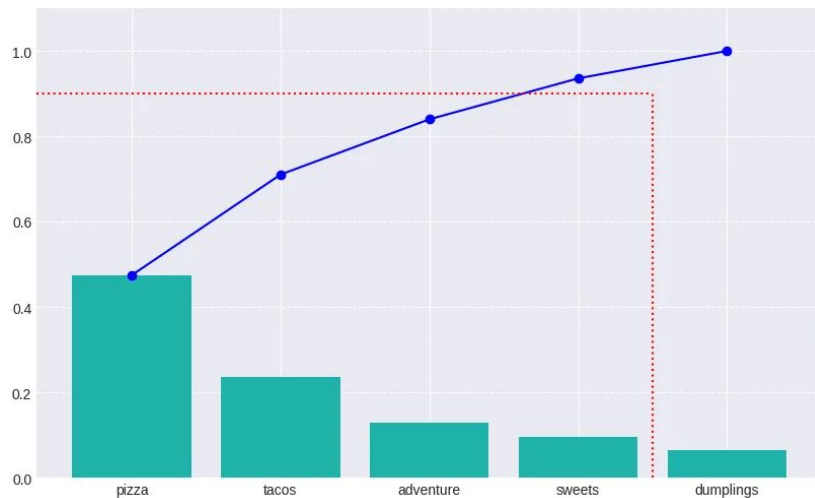
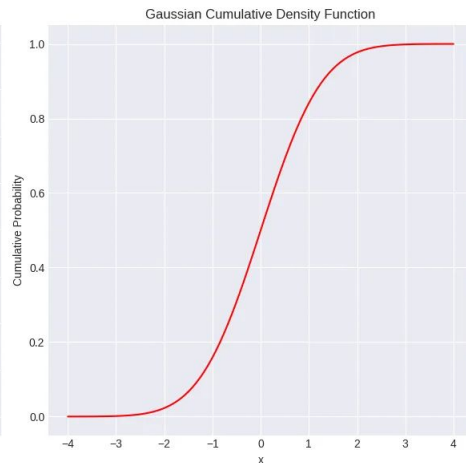
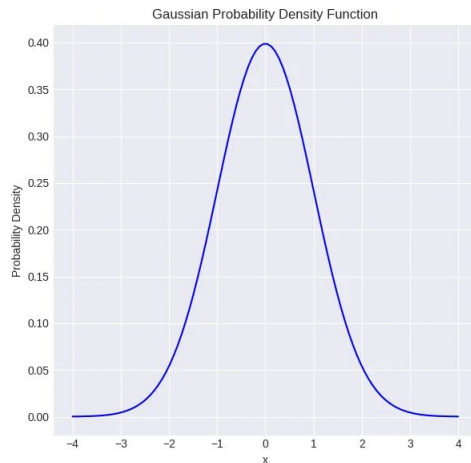
Top k

- При генерации модель рассматривает только k токенов с наибольшей вероятностью. Затем вероятности этих токенов нормализуются, и из них семплируется выходной токен.



Top p

- Из модели приходят вероятности всех токенов;
- Начиная с самого вероятного токена, считаем кумулятивную вероятность и останавливается, когда она превышает p ;
- Нормализуем вероятности внутри получившегося ядра (nucleus) и семплируем выходной токен.



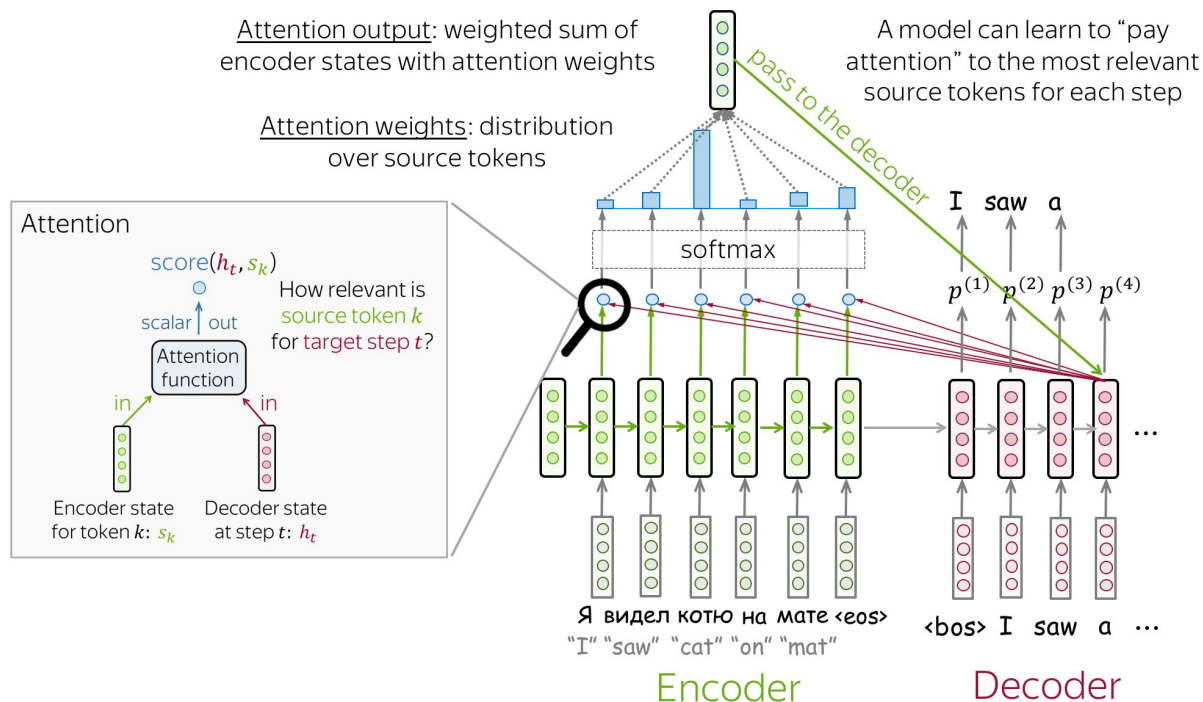
Недостатки seq2seq-подхода

- Модель забывает начало предложения, особенно если оно длинное;
- Хотелось бы на каждом этапе декодирования иметь возможность “вспомнить” изначальную последовательность.

Attention

Решение: механизм attention

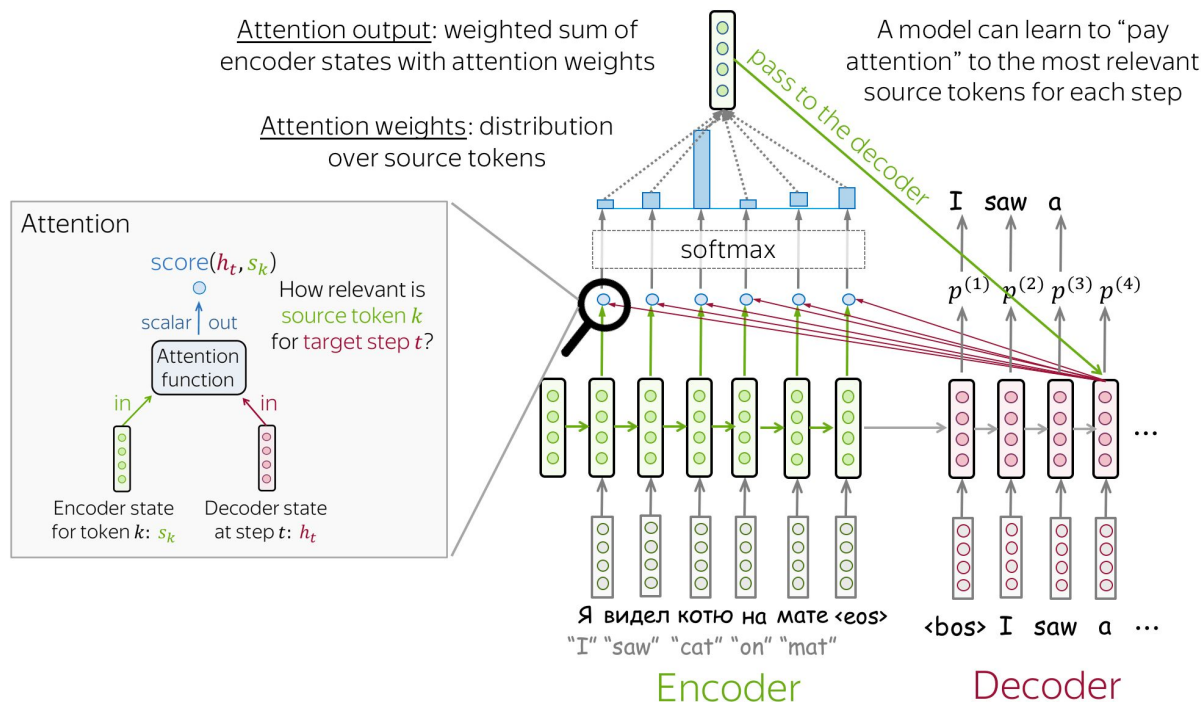
Идея: нужно обратить внимание на входную последовательность перед тем, как генерировать очередной token выходной последовательности.



Решение: механизм attention

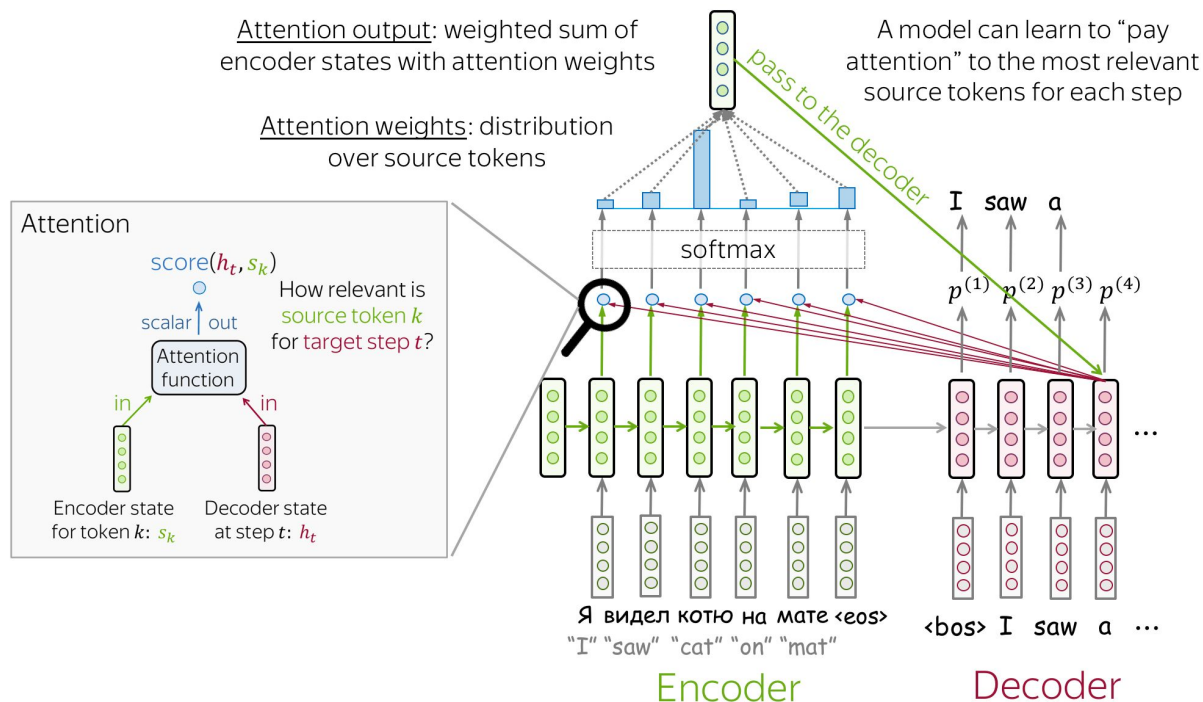
Каждый раз перед генерацией какого-нибудь слова мы будем вычислять “важность” $s(h_t, s_k)$ каждого входного токена для текущего состояния декодера.

Например, когда мы готовимся сгенерировать слово “cat”, хорошо обученная модель должна обратить больше внимания на слово “котя”.



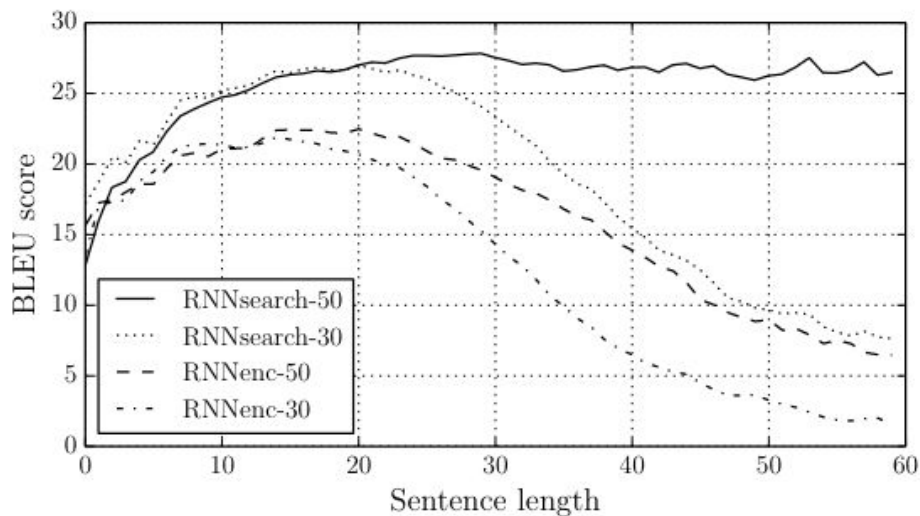
Решение: механизм attention

Механизм внимания получает на вход текущее состояние декодера и все состояния энкодера. На выходе получают веса внимания (“важность” токенов входной последовательности для предсказания) и выход внимания: сумму произведений весов внимания на состояния энкодера, которая передается обратно в декодер.



Плюсы и минусы seq2seq с attention

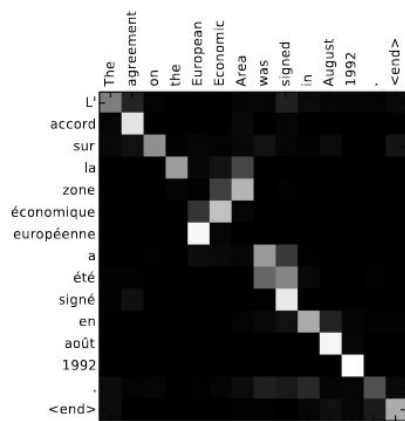
- + Теперь мы можем “запоминать” контекст любой длины => качество решения задач увеличивается;
- + Внимание интерпретируемо.
- Вычисления всё ещё могут быть только последовательные.



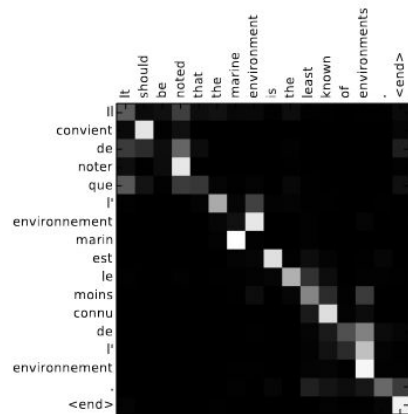
Визуализация attention

Внимание может быть визуализировано, а также использовано для элайнмента (выравнивания) по словам.

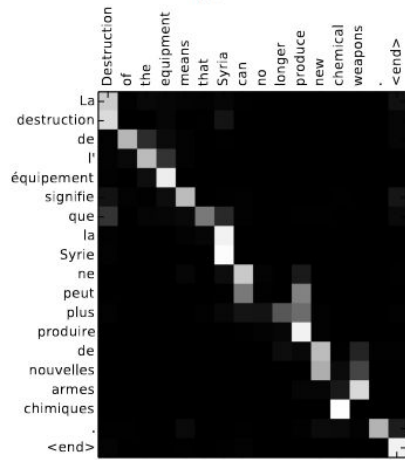
На картинках справа можно видеть, на какое слово из исходной последовательности (по оси x) модель обращала больше всего внимания, когда генерировала слово из результирующей последовательности (по оси y).



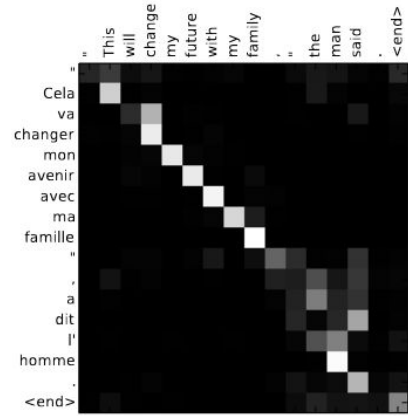
(a)



(b)



(c)



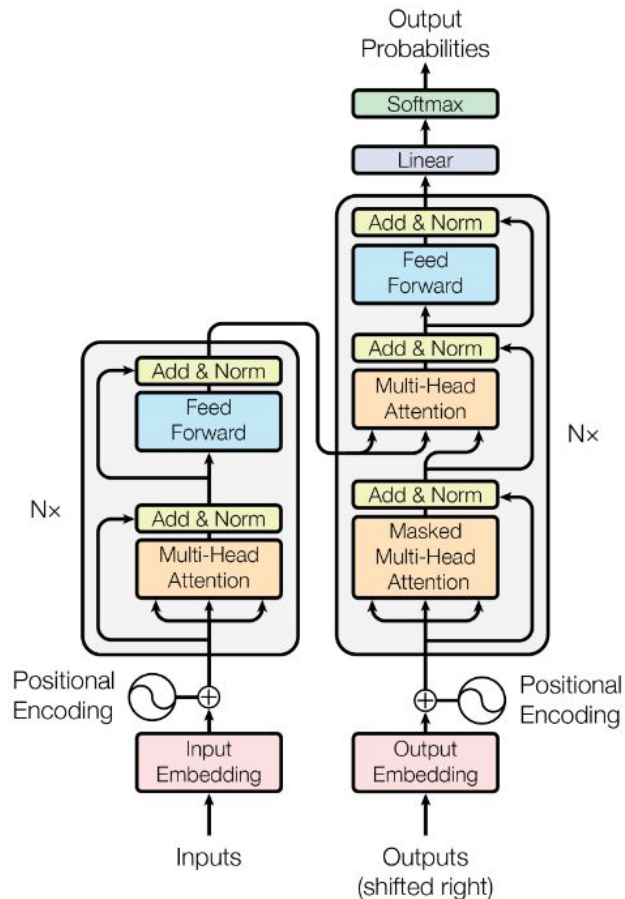
(d)

Transformers

Трансформеры

Идея: применять архитектуру внимания без рекуррентной нейросети.

Трансформеры все еще используют энкодер и декодер, но некоторые вычисления уже могут происходить параллельно.



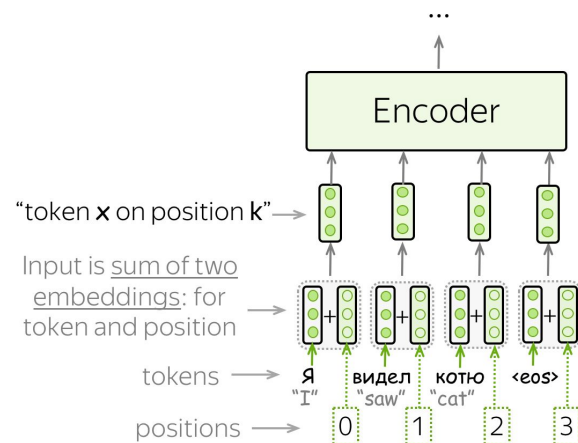
Positional encoding

Идея: добавляем к каждому слову вектор, который дает модели информацию о позиции слова в предложении.

Существуют разные подходы к созданию positional encoding. Один из самых популярных выглядит так (pos - позиция токена в предложении, i - dimension index):

$$p_{pos,2i} = \sin\left(\frac{pos}{10000^{\frac{2i}{embedding_dim}}}\right) \quad p_{pos,2i+1} = \cos\left(\frac{pos}{10000^{\frac{2i}{embedding_dim}}}\right)$$

Данный метод помогает модели запоминать относительные позиции токенов в предложении.



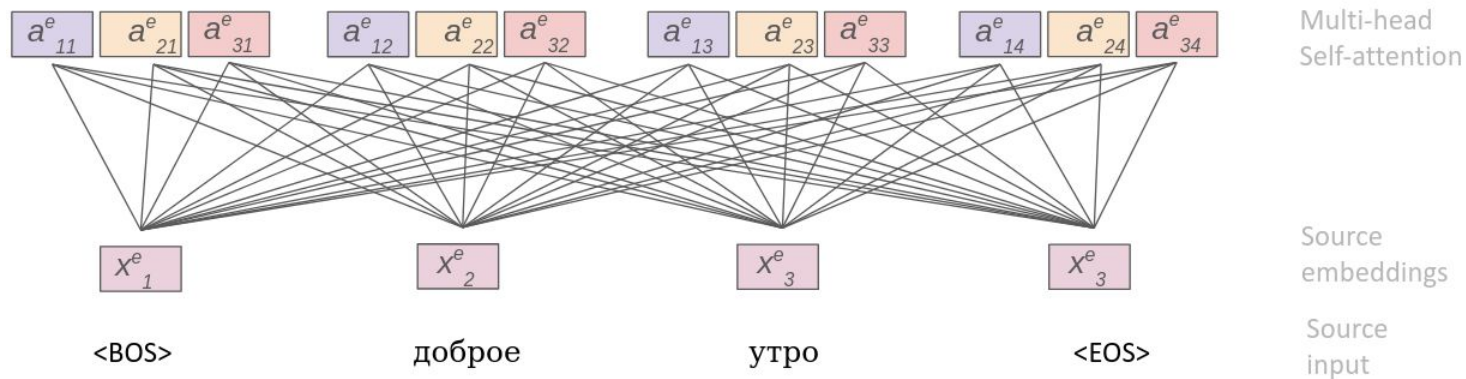
Multi-head self-attention

- Self-attention: каждое состояние смотрит на все остальные состояния. Нужно для того, чтобы посмотреть на каждый token в контексте исходного предложения.

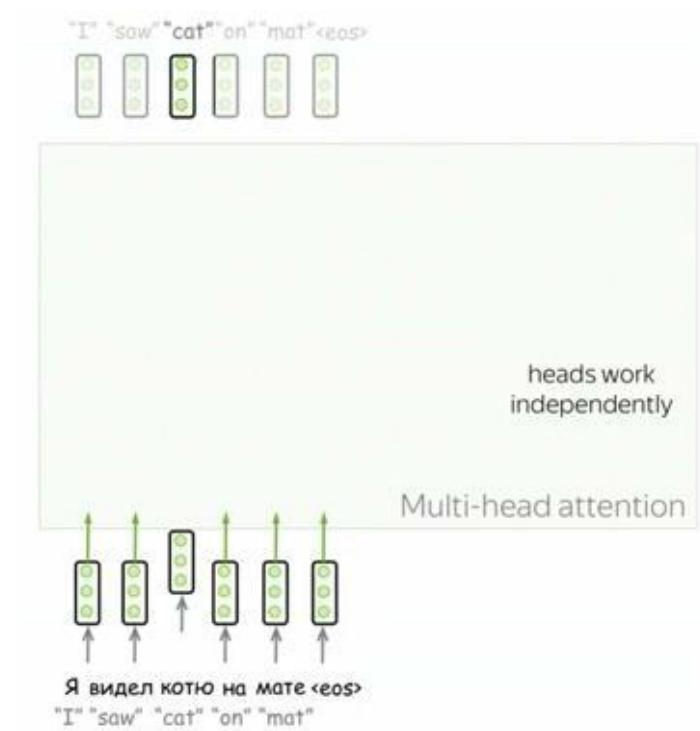


Multi-head self-attention

- Multi-head attention: смотрит на зависимости разного типа между словами в предложении. Разные головы обращают внимание на разные токены.

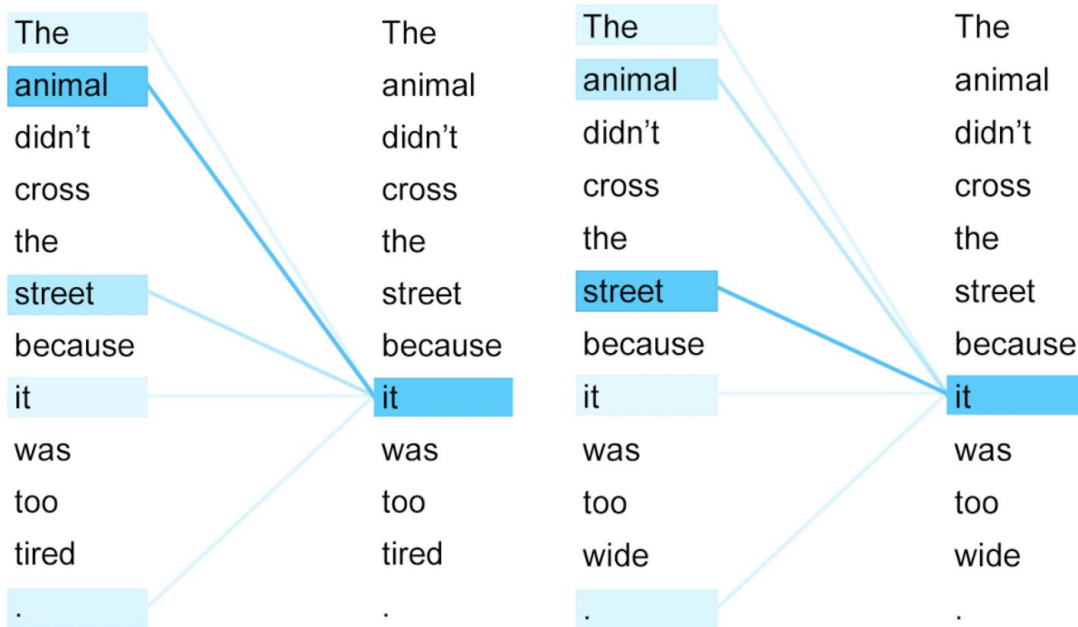


Multi-head self-attention



Multi-head self-attention

Итог применения
multi-head self-attention:
учтен контекст слова,
снята омонимия, учтены
синтаксические связи в
предложении.



Masked multi-head attention

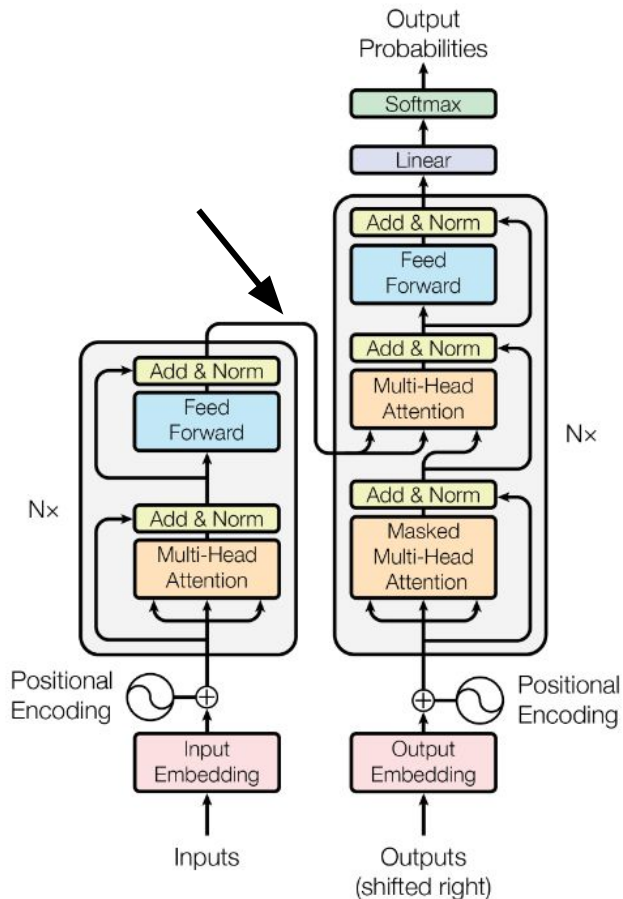
Masked multi-head attention используется в декодере.

Во время генерации новой последовательности декодер не может смотреть на токены, которые он еще не сгенерировал (даже при обучении). Поэтому attention-блоки декодера связываются только с эмбедингом текущего входного токена и предыдущих.

Cross-attention

Attention-векторы, принимающие финальные состояния энкодера (единственное место связи энкодера и декодера в трансформере), называются cross-attention-векторами.

Каждый такой вектор вычисляется на основе эмбединга декодера и всех эмбедингов энкодера. Т.е. в этот момент нейросеть смотрит на генерируемое слово с учетом всего входного предложения.



QKV Attention

Query, Key, Value Attention - это способ вычисления внимания для вектора y , исходя из векторов $x_1 \dots x_i$. Так вычисляется self-attention score.

В данном случае y - вектор запроса, x - ключевые вектора. Ключевые вектора содержат информацию, которая может быть добавлена во внимание вектора запроса. Таким образом, токены, составляющие контекст текущего токена, могут добавить информацию в его вектор внимания.

QKV

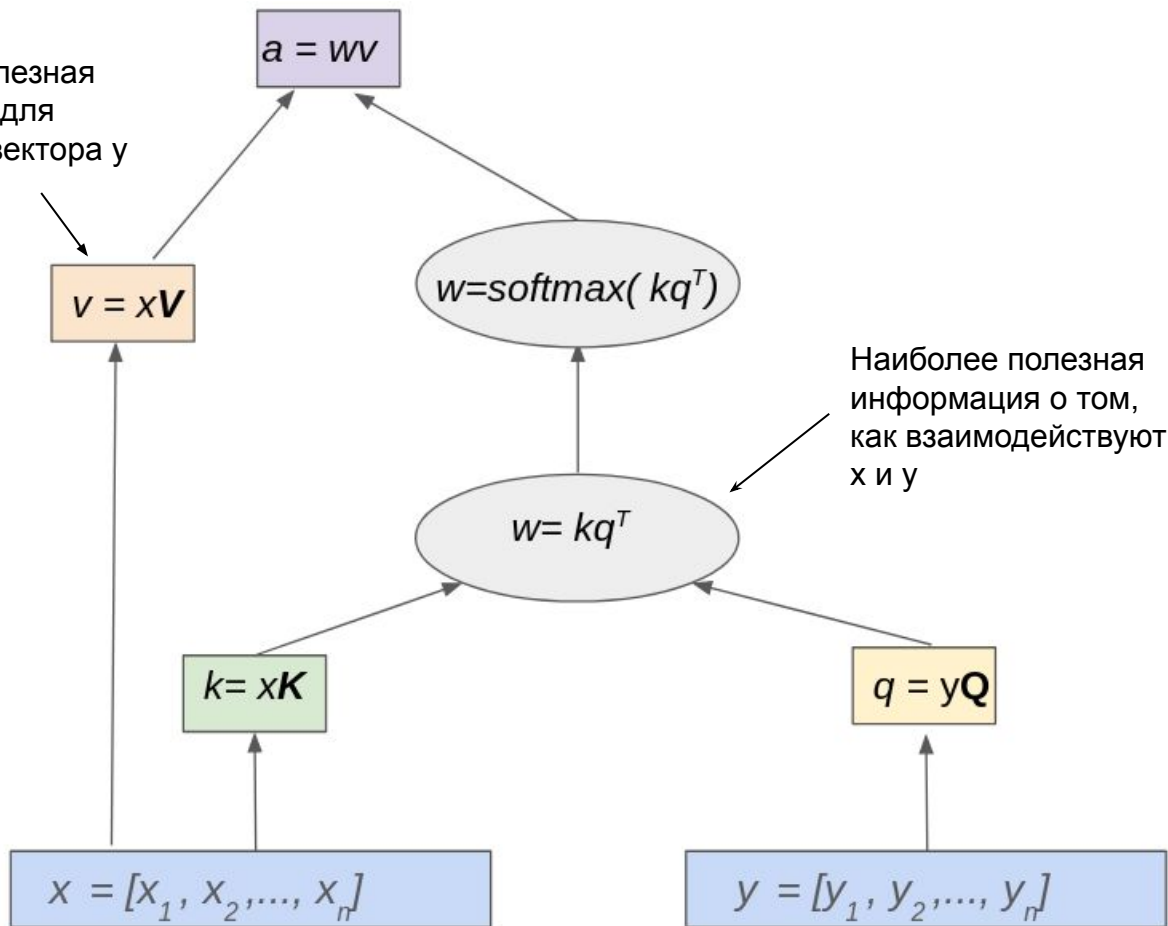
Q: вектор, который “ищет информацию” о текущем токене;

K: вектор, который содержит потенциальную информацию из контекста;

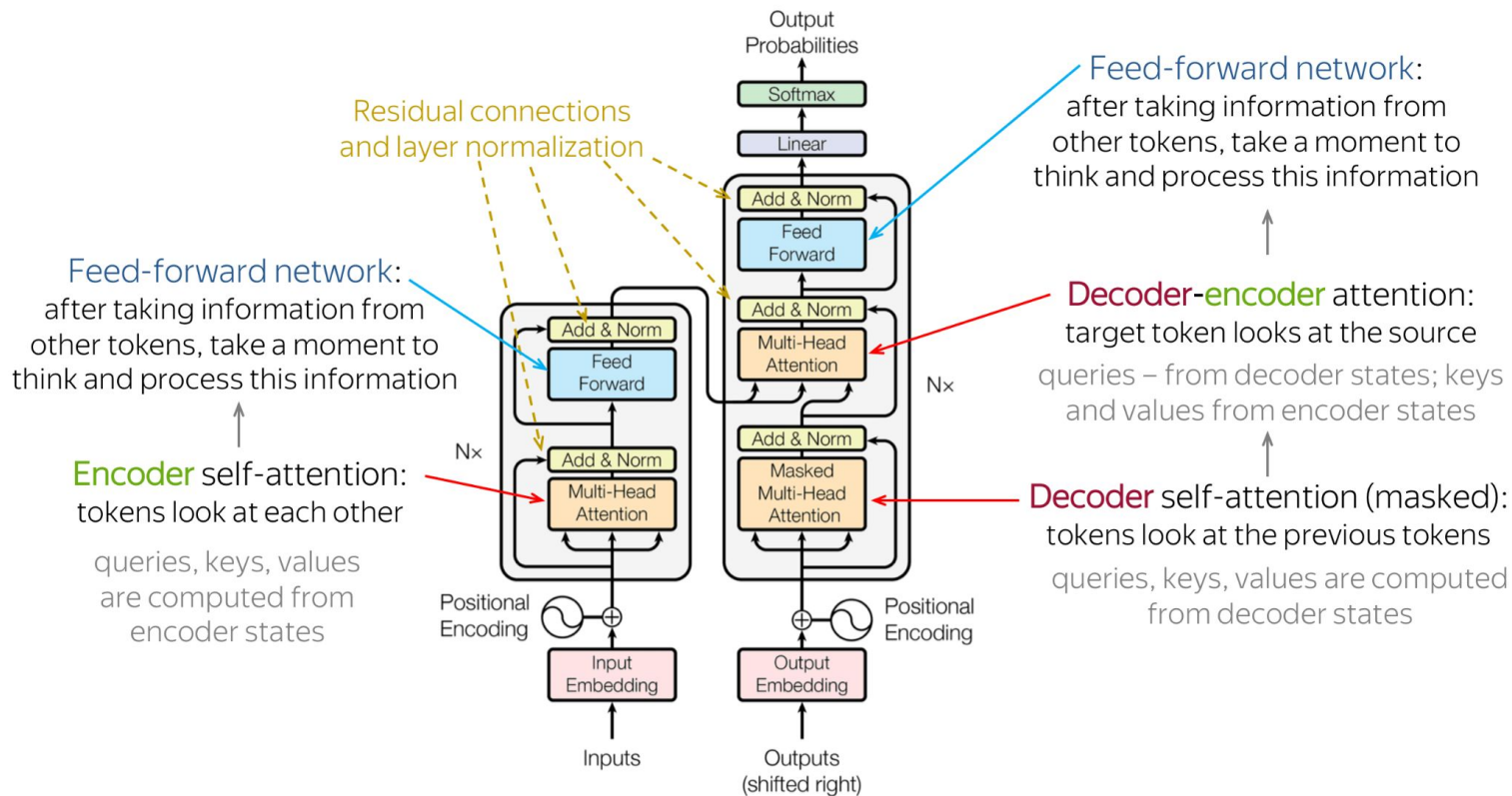
V: взвешенная сумма, которая определяет, какую информацию нужно отдать.

Эти матрицы также обучаются в процессе обучения трансформера.

Наиболее полезная информация для обновления вектора y



Layer norm, feed-forward blocks, residual connections



Layer norm, feed-forward blocks, residual connections

- FFNN: два линейных блока с ReLU между ними. Исследует информацию, пришедшую из attention-блоков.
- Residual connections (“Add” в Add&Norm): добавляет входную информацию из блока к его выходу. Позволяет сохранять информацию при переходах между слоями и эффективно углублять нейросеть.
- Layer norm (“Norm” в Add&Norm): нормализует вектора токенов по отдельности. Стабилизирует обучение и улучшает сходимость.

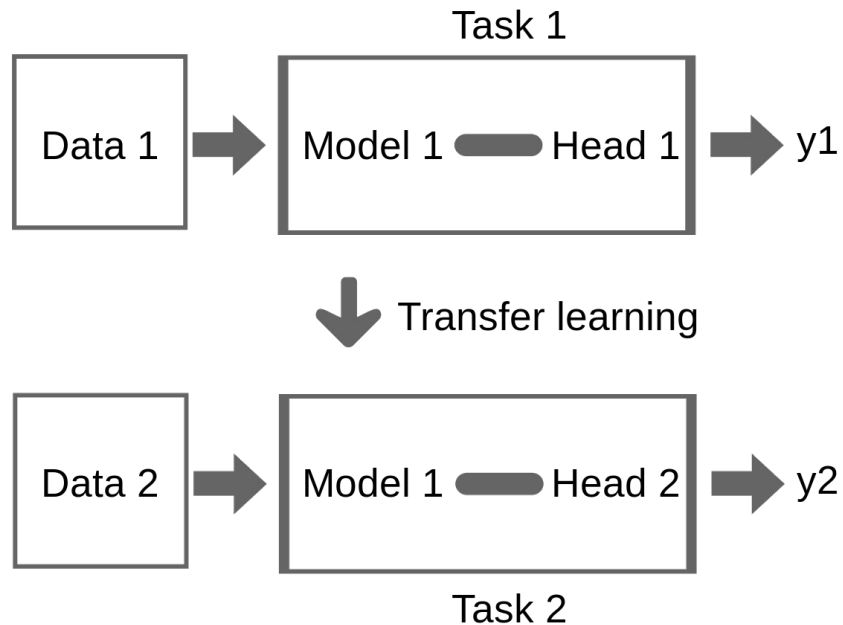
Fine-tuning

Дообучение/тонкая настройка - это техника, при которой модель, обученная на каком-то датасете, продолжает обучение на других данных. Дообучение предполагает изменение каких-то параметров модели: либо всех, либо только некоторых (в этом случае неизменяемые параметры называют замороженными).

Дообучение больших моделей, как правило, происходит путем сохранения изначальных параметров и добавления специального слоя, обучаемого для конкретной задачи.

Transfer learning

Transfer learning - это техника в машинном обучении, которая подразумевает обучение модели для решения нескольких задач. Таким образом, знания, полученные моделью в ходе тренировки на одной из задач, могут быть использованы для решения другой. Например, знания модели, умеющей делать саммаризацию, могут быть полезны при дообучении этой модели для задачи упрощения текстов.



Основные типы трансформеров

BERT

Bidirectional Encoder Representations from Transformers (BERT) - это двунаправленный трансформер, состоящий (в оригинальной имплементации) из слоя эмбеддингов, слоя трансформерных энкодеров и декодирующего слоя, превращающего выученные представления обратно в слова.

Первичное обучение BERT происходит на двух задачах: языковом моделировании и предсказании следующего предложения. BERT можно с высокой эффективностью дообучить на задачи классификации, natural language inference (см., например, задачу entailment), gap resolution и некоторые другие.

BERT имеет несколько модификаций, например, [RoBERTa](#), [ALBERT](#) и др.

BERT: обучение

Языковое моделирование: предсказание токена по контексту. Токен может быть:

- Заменен маской [MASK] с вероятностью 80%,
- Заменен случайным словарным токеном с вероятностью 10%,
- Не заменен с вероятностью 10%.

Предсказание следующего предложения:

- Для предложений "[CLS] my dog is cute [SEP] he likes playing" выдать токен [IsNext];
- Для предложений "[CLS] my dog is cute [SEP] how do magnets work" выдать токен [NotNext].

GPT

Generative pre-trained transformers (GPT) - односторонний (decoder-only) трансформер, натренированный на большом корпусе текстов.

Первичное обучение происходит на задаче предсказания следующего слова, затем сеть можно дообучать для более специфических задач.

GPT-подобные модели обычно дообучаются для задач генерации текста.

В настоящее время существует пять поколений моделей GPT.

Sequence-to-sequence-трансформеры

BERT состоит только из энкодера трансформера, а GPT - только из декодера. Однако, есть трансформеры, использующие и то, и другое, например:

- BART - хорошо подходит для суммаризации и упрощения;
- T5 - подходит для большинства sequence-to-sequence задач, таких, как перевод, суммаризация, перефразирование и т.п.

Что можно сделать при помощи
трансформеров?

(Практически всё)

Классификация текста целиком

- Поиск спама, поиск автоматически сгенерированного текста;
- Анализ тональности;
- Классификация нескольких предложений: entailment, может ли предложение А следовать за предложением В и т.д.

Пример: [The Stanford Natural Language Inference \(SNLI\) Corpus](#)

Premise: Two boys are playing outside. **Hypothesis:** There are kids outside.

Classification: Entailment

Premise: A young boy sits on the shoulders of a woman, who is carrying a young girl.

Hypothesis: A woman is driving her kids around. **Classification:** Contradiction

Классификация текста целиком

- Acceptability: определение “правильности” предложения.

Пример для русского языка: [RuCoLa](#)

id	sentence	acceptable	error_type	detailed_s
36	Все, кто учились в математическом классе, сдали тест на отлично.	0	Syntax	USE8
37	Его отсутствие на собрании подтвердило плохое мнение.	1	0	Paducheva2004
38	Он бежит в саду.	1	0	Testelets
39	У его веры был рациональный характер.	0	Semantics	Seliverstova

Классификация каждого слова в предложении

- POS-tagging;
- Named Entity Recognition;
- Определение синтаксических ролей.

Примеры готовых решений: spacy, stanza, [natasha](#)

Примеры датасетов для грамматики и синтаксиса: [Universal Dependencies](#) (включает [SynTagRus](#)), [Penn Treebank](#)

Примеры датасетов для выделения именованных сущностей: [FactRuEval](#)

Классификация - пример мини-проекта

- Статья про RuCoLa: <https://aclanthology.org/2023.bsnlp-1.15.pdf>
- Гитхаб: <https://github.com/upunaprosk/la-tda/blob/master/>
- Проектная часть: повторить эксперименты авторов с другой моделью
 - LLM на ограниченной подвыборке примеров из тестового множества - бонус, если сами отберете наиболее сложные примеры;
 - Более современной BERT-моделью для русского, которую вы обучите на тренировочных данных. Не обязательно повторять все шаги обучения авторов, тренировка может быть любой, главное - использовать тот же тестовый сет и ту же метрику, что в статье

Генерация текста

- Заполнение пропусков в тексте;

Пример датасета: [AGRR - соревнование по разрешению эллипсиса](#)

- Генерация по заданию (промпту);

Пример датасета: [Writing Prompts](#)

- Генерация ответов на вопросы;

Пример датасета: [SQUAD: The Stanford Question Answering Dataset](#)

- Генерация нового текста на основе входного: машинный перевод, рерайтинг: суммаризация, детоксификация и т.п.

Машинный перевод

- Мультилингвальный

Пример датасета: любая часть [OPUS](#), например, [Open Subtitles](#)

- Монолингвальный

- Упрощение: выходной текст должен быть проще, чем исходный

Примеры датасетов: [RuSimpleSentEval](#), [MultiSim](#)

- Парафразы: выходной текст должен быть изменен с сохранением смысла

Пример датасета: [ParaPhraser.ru](#)

Рерайтинг

- Детоксификация

Задача: переписать “токсичное” предложение “нетоксично”.

Пример датасета: [RUSSE Detox](#)

- Саммаризация

Abstractive summarization: генерация саммари из новых предложений, отражающих суть изначального текста. Extractive summarization: извлечение предложений из изначального текста, составляющих его краткое содержание.

Пример датасета: [CNN/Daily Mail](#)

Генерация - пример мини-проекта

- Статья: Dementieva et al., RUSSE-2022: Findings of the first russian detoxification shared task based on parallel corpora
- Гитхаб соревнования: https://github.com/s-nlp/russe_detox_2022
- Варианты проектной части:
 - Решить соревнование новым методом или модификацией одного из методов, примененных участниками;
 - Взять лучший метод решения и применить его на другом датасете для детоксификации - например, <https://huggingface.co/textdetox>.

Практика 1 - упрощение текстов моделью T5:

https://colab.research.google.com/drive/1qHsmxn7w_r9TKX805vlqRwi0uuWxo-fY?usp=sharing

Практика 2 - POS-теггинг при помощи DistilBERT:

<https://colab.research.google.com/drive/1PyoZJMkxWTvzNTbok2DiL9GctZg6tAqc?usp=sharing>

Источники:

- Лекции Татьяны Гайнцевой:
 - https://youtu.be/QjpEWRq-Cak?si=T1a0yaB7z_xZ_QPG
 - <https://youtu.be/Fki-Xe3CGg8?si=m7-rJOGhLL0fiK5f>
 - https://youtu.be/TBEwpgyoo20?si=_13SwxQTguz9x3vq
 - <https://youtu.be/tsee8mosj5U?si=uBvsfR3RmSjir1SV>
- Блог Лены Войта:
https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html
- Давид Дале об обучении T5 перифразированию:
<https://habr.com/ru/articles/564916/>